Kestrel v3

An offline personal assistant on a laptop

Project report

Kestrel answers in English and Hinglish, keeps tasks and reminders, finds and answers from the user's notes, writes emails, rewrites and translates text, chats, and writes small Python programs that it checks in a sandbox. All of it runs on one laptop, without the internet and at no cost.

[Author name]
[Affiliation]

28 October 2026 · version tag `v3-frozen`

Contents

1 Summary **2**

2 Goals and starting point **2**

3 What Kestrel v3 does **3**

4 How it works **4**
4.1 The service 4
4.2 Deciding the kind of answer 5
4.3 Notes by meaning and answers from notes 5
4.4 Code in a sandbox 5
4.5 Two changes that made the service fast to start 6

5 How it was built **6**

6 Results **7**
6.1 Choosing the kind of answer 7
6.2 Finding notes by meaning 7
6.3 Answering from the notes 8
6.4 Code 8
6.5 Acceptance run 8

7 Known limitations **9**

8 Installing and using it **10**

9 What comes next **10**

A Decision record **10**

B Where the numbers come from **10**

1 Summary

Kestrel is a personal assistant whose every step runs on the user's own laptop. Version 1 (v1) was a research system that sorted requests into nine intents: eight app actions (search, open or summarize notes, add or delete a task, a preference, what it remembers, small talk) and *unsupported*, which it refused. Version 2 made it usable every day: tasks with times, reminders, an always-on service and a browser page. Version 3 (v3), reported here, set out to make Kestrel answer the requests it used to refuse, on the same laptop and still completely offline:

- tasks and reminders (from v2), in English and Hinglish;
- finding notes by meaning, not only by their words, and summarizing them;
- answering questions from the user's notes, and saying so when the notes do not answer;
- drafting emails, rewriting and translating text, and normal chat;
- writing and fixing small Python programs, checked in a sealed sandbox, and explaining code.

All of it runs on the CPU of a 6-core laptop with 14.95 GiB of memory, with two models already used in v1: a small multilingual sentence encoder and the 4-bit Gemma-4-E2B language model. Table 1 gives the main results; each comes from a log in the repository.

Table 1: Main results of v3. Test splits were run once, after every setting had been chosen on development data. All evaluation sets are small and synthetic (written for this project, not by real users).

What was measured	Result
Choosing what kind of answer a request needs (test, $n = 280$)	256/280 right
... on the 24 test requests never scored before	23/24 right
a skill started that the request did not ask for	6 times
Finding the right note by meaning, when keywords find nothing (test)	14/18
a wrong note added to a query with no note	0/2
Answering questions from the notes (test)	19/19 answered right
questions the notes do not answer, declined	6/6
Small Python requests (test): functions, fixes, explanations[a]	6/6, 6/6, 6/6
scripts that read input	3/6
Memory the system cannot take back, language model loaded	3247 → 1144 MiB
Time until the language model is ready to answer	19.1 → 2.0 s
End-to-end acceptance run (real service, typed requests)	34/36 passed
Browser page checks, headless and on screen	8/8, 8/8
Automated tests (Rust, Python)	448, 510 passed

[a] Measured with the evaluation's own explain prompt; in the service, explaining code is a chat answer.

The version was frozen on 27 October 2026 after the acceptance run, with 12 known limitations written down (Section 7). The two most important: some questions about the user's own life are not recognised as notes questions and are refused, and scripts that read input fail half the time.

2 Goals and starting point

Where v3 started. After v2, Kestrel could add, list, finish, rename and reschedule tasks, remind at a time, search, open and summarize notes, remember preferences, and run as an always-on service with a browser page. A request outside these, such as *"write an email to my landlord"* or *"translate this into Hindi"*, usually got `I can't do that`, or ended in a wrong action.

The goal (decided 26 October 2026). Achieve, on this laptop and completely offline, with good latency and memory use: tasks and reminders; notes search by meaning and summaries;

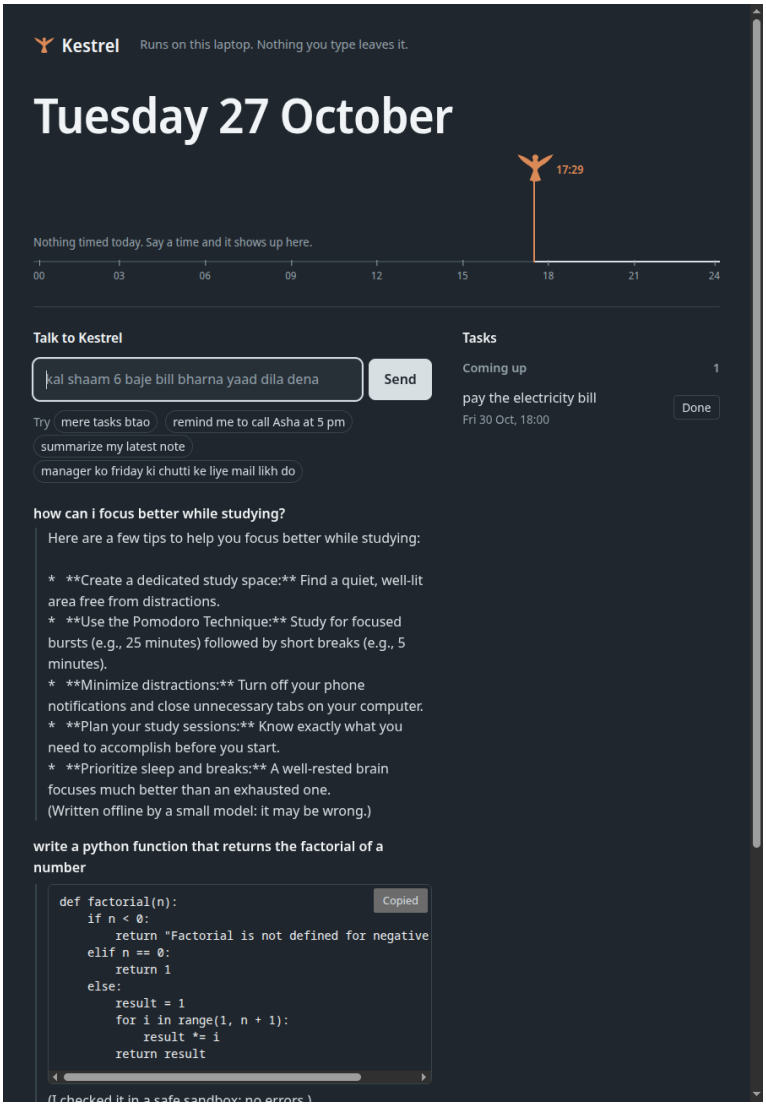


Figure 1: Kestrel's browser page during the acceptance run (headed Chrome on the author's screen): the day strip with the kestrel marking the current time, the request box with example requests, the task list, a chat answer with its caveat, and a code answer in a code box with a Copy button and the sandbox's check below it.

email drafts in English and Hinglish; rewrite and translate; normal chat; questions answered from the notes; and, with a compile and test loop, small Python scripts, single functions, explaining code and fixing simple errors. A bigger machine is the next version's target (v4).

Constraints. No cloud service, no paid model or API, no new model download (only models already on the laptop), no change to the frozen research system of v1, and a laptop with 14.95 GiB of memory shared with everyday use. On this laptop, Gemma-4-E2B writes about 12.1 tokens per second and reads a prompt at about 58 tokens per second, so every design choice had to keep prompts short and the model unloaded when idle.

3 What Kestrel v3 does

The examples below are real requests and answers from the acceptance run (Section 6.5), copied from its log. That run used a scratch copy of the service with sample notes, not the author's own notes.

Tasks and reminders. *"kal shaam 6 baje bill bharna yaad dila dena"* → `created task #2: bill bharna (due 2026-10-28 18:00)`. *"move task 2 to friday"* → `task #2 now due 2026-10-30`. Deleting asks first. A timed task pops up as a desktop notification at its time.

Notes by meaning. The keyword search still runs first. When it finds fewer than three notes, the note closest in meaning is added if it is close enough: *"search my notes for maa ke liye tohfa"* → `found 1 note: - 2026-09-08-mom-birthday.md (by meaning)`, although the keyword search, which needs every word of the query in a note, found nothing.

Questions answered from the notes. *"maa ka birthday kab hai?"* → `Maa ka birthday 20 September hai. (from 2026-09-08-mom-birthday.md)`. When the notes do not say: *"goa wale hotel ka naam kya hai?"* → `The notes do not say this: so the reply is not available for this information`. An answer is shown only if every number and name in it occurs in the notes.

Emails, rewriting and translation. *"manager ko friday ki chutti ke liye mail likh do"* → a Hinglish draft starting `Subject: Friday ki chutti ke liye request`. *"make this polite: send me the file now"* → `Could you please send me the file now?` *"translate to english: aaj bahut garmi hai"* → `It is very hot today`. Translation into Hindi is written in Devanagari script; Hinglish stays in Latin letters, although a request to translate into Hinglish often came back in English (1/4 on development items).

Chat. General questions, explanations, ideas and poems are answered by the language model, and every such answer ends with `(Written offline by a small model: it may be wrong.)`. Questions that need live data or the internet (*"what's the weather in delhi today?"*) are refused.

Code. *"fix this function: def add(a, b): return a - b"* → the fixed function, then `(I checked it in a safe sandbox: no errors.)` and `(Written offline by a small model: try it before you rely on it.)`. On the page, code appears in a box with a Copy button.

Saving to notes. *"Buy rice, dal and oil on Saturday from the market. add this to notes"* asks `Save this as a new note, 2026-10-27-buy-rice-dal-and-oil-on-saturday-from.md (10 words)?` and, after a yes, saves the text exactly as typed. It never writes over an existing file.

Where to use it. In the browser page (Figure 1), opened with `kestrel open` or from the applications menu, or in a terminal with `kestrel chat` and `kestrel say`. Answers from the language model appear as they are written, except answers from the notes and note summaries, which appear once they have been checked.

4 How it works

4.1 The service

One program, `kestrel serve`, runs from login and keeps the cheap parts loaded: keyword rules and the sentence encoder (multilingual-e5-small). The language model, Gemma-4-E2B at 4-bit, runs in a separate worker process that starts on the first request that needs it and stops after 10 idle minutes, which gives its memory back. The terminal client and the browser page talk to the service over a local socket and a local web server on 127.0.0.1 that needs a key.

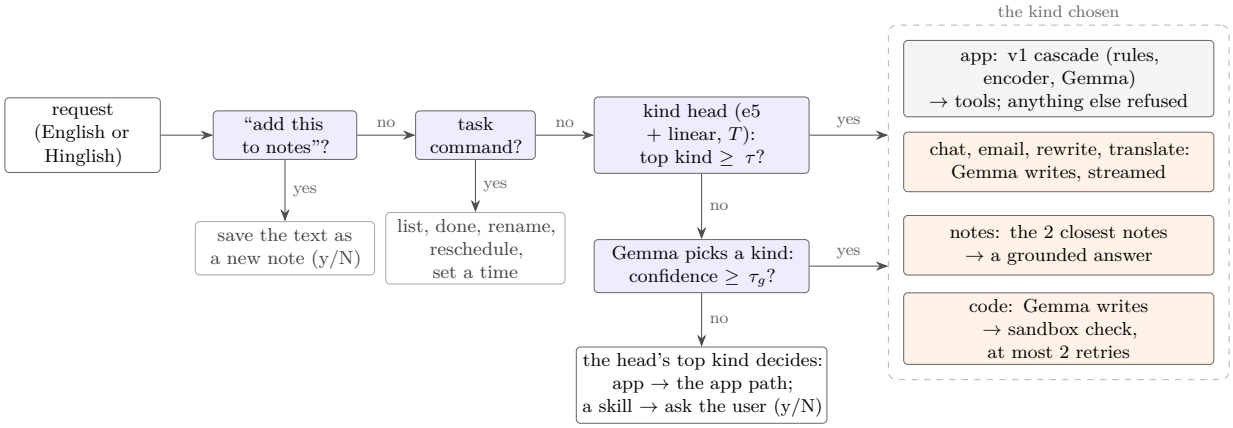


Figure 2: The path of a request. Saving a note and task commands are recognised first. Then a small, fast classifier decides the kind of answer: the old app path, or one of the new skills. When it is unsure, the language model decides; when that is unsure too, Kestrel proposes a skill as a yes/no question, or takes the old path if the request leans to the app.

4.2 Deciding the kind of answer

Every request that is not a task command first gets a *kind*: *app* (the v1/v2 path: tasks, notes search and summaries, memory, small talk, refusals), *chat*, *email*, *rewrite*, *translate*, *notes* (a question about the user's own life) or *code*. The encoder turns the request into a vector, and a small trained classifier on top gives a probability for each kind. Its probabilities are calibrated (temperature scaling): fitted so that its confidence stays close to how often it is right on held-out data. Then:

1. if the top kind reaches the threshold $\tau = 0.8$, it decides;
2. otherwise Gemma reads the request with a short description of each kind and chooses one; if its calibrated confidence reaches $\tau_g = 0.7$, its choice decides (about 523 ms on this laptop);
3. otherwise, if the classifier's top kind is a skill, Kestrel asks, for example `Write this as an email or message draft?`; if it is app, the request takes the old path.

The thresholds were chosen on development data by a rule written down before the run: fewest wrong decisions, then fewest questions to the user.

4.3 Notes by meaning and answers from notes

Each note is cut into chunks (its title and one paragraph), and each chunk becomes a vector once; the vectors are cached and recomputed only for notes that change. A search query becomes a vector too, and the closest note is added if it is close enough (cosine similarity at least $\theta = 0.83$). The encoder does not understand Romanized Hindi well, so a query with common Hindi words is first put into English by Gemma (*"maa ke liye tohfa"* became *"gift for mother"*). For a notes question, the two closest notes go to Gemma with the instruction to answer only from them, in the question's language, or to say exactly `NOT IN NOTES`. A grounding check then rejects an answer whose numbers or names are not in the notes.

4.4 Code in a sandbox

For a code request Gemma writes standard-library Python. The code then runs inside bubblewrap, a Linux sandbox: no network, no access to the home folder, a read-only system, an empty temporary folder, and limits of 512 MiB of memory, 10 s of processor time, 10 MiB files and 64

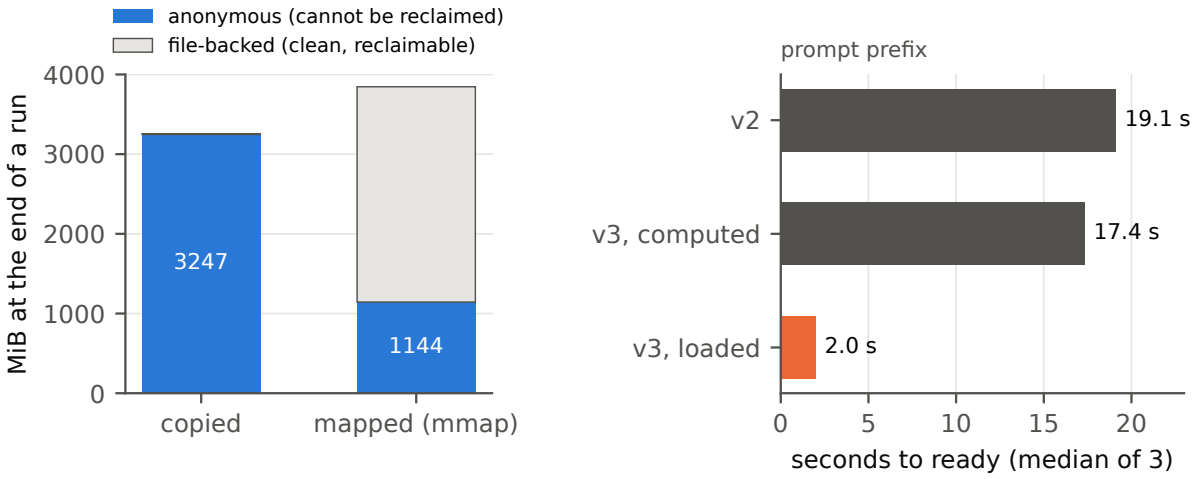


Figure 3: Left: memory at the end of a run with the model's weights copied or mapped; the grey part is the mapped file, which the system can drop when it needs memory. Right: seconds until the language model's worker is ready.

processes. Code is imported (so it compiles and its top level runs); code that waits for input is only compiled, since the sandbox cannot type for it. A syntax error, a crash or a timeout is sent back to Gemma, at most twice, with the error. The answer always says what was checked.

4.5 Two changes that made the service fast to start

- **Memory-mapped weights.** The model file is mapped into memory instead of copied. The memory the system cannot take back fell from 3247 to 1144 MiB (64.8% less), and loading became faster (3656 to 2477 ms from a cold disk cache).
- **A saved prompt.** Before answering, the worker reads a 1132-token prompt of examples. v3 saves the model's state after reading it and loads that state next time. The worker became ready in 2.0 s instead of 19.1 s (medians of 3 starts).

5 How it was built

Decisions on record. Every design choice of v3 was recorded as a decision with its context, the options considered and the reason (thirteen records, from the goal on 26 October 2026 to the freeze on 27 October 2026; Appendix A). The author made each decision, mostly by accepting a stated recommendation.

Technology. The assistant is written in Rust (a workspace of crates; the v3 parts live in the command-line app, and the memory-mapped load and the saved prompt in the inference crate), and runs the models through llama.cpp. Evaluation scripts are Python. The sandbox is bubblewrap, and the browser page is a single self-contained HTML page served by Kestrel itself, tested with Google Chrome over its DevTools protocol. No paid service was used, and only models already on the laptop were used.

Data and fair testing. Each part was measured on a small evaluation set written for it (Table 2), in English and Hinglish pairs, grouped into *families* of similar requests. Each set was split by family into development data, used to choose every setting, and a test split, run once at the end. Most sets were reviewed card by card by the author on a review page; for the code set

and the third kinds set the author waived the review, and they were reviewed by the AI assistant that drafted them, with a separate agent checking the files against written criteria.

Table 2: Evaluation sets. All are synthetic and split by family.

Dataset	Labels / content	Items (train / cal / val / test)	Review
Kestrel-Kinds-v1	6 kinds	920 / 115 / 136 / 256	reviewed
Kestrel-Kinds-v2	7 kinds (+ code)	1000 / 123 / 152 / 280 (24 fresh)	claude reviewed
Kestrel-NotesQ-v0	18 fixture notes	dev 22 (4 none) / test 20 (2 none)	reviewed
Kestrel-NotesQA-v0	same notes	dev 26 (6 unans.) / test 25 (6 unans.)	reviewed
Kestrel-CodeTasks-v0	4 types	dev 24 / test 24 (24 families)	claude reviewed

6 Results

6.1 Choosing the kind of answer

On Kinds-v2's test split, 256/280 decisions were right (91.4%); 13 requests were asked about, 5 skill requests were sent the old way (the app path), and 6 times a skill was started that the request did not ask for: three jokes and a web search answered as chat, a time question sent to notes, and a loop fix sent to rewrite. English and Hinglish were close (130/140 and 126/140). Without the language model's second opinion, only 221/280 were right: more requests were asked about (37) or sent the old way (20).

256 of these 280 test requests had already been used once, to test the previous version of the kinds set, and the new training families were written after its mistakes were known; so this result is optimistic. On the 24 fresh requests, 23/24 were right. Per kind (Table 3), chat and translation were always recognised, code 7/8, email 15/16 and notes questions 12/16; rewrite requests only 8/20, most of the rest being asked about first.

Table 3: Right decisions per kind on the two test splits. Kinds-v1 had no code kind.

	Kinds-v1 test		Kinds-v2 test	
Gold kind	head alone	head + Gemma	head alone	head + Gemma
app	179/180	180/180	181/184	178/184
chat	2/16	11/16	6/20	20/20
code	–	–	1/8	7/8
email	12/16	13/16	12/16	15/16
notes	1/12	1/12	7/16	12/16
rewrite	3/16	4/16	6/20	8/20
translate	9/16	11/16	8/16	16/16

6.2 Finding notes by meaning

Every query in this set misses its note by keywords, so every find is the meaning search's. Before the translation step, only 1/9 Hinglish development queries found their note; with it, 9/9. On the test split the right note was added for 14/18 queries (7/9 English, 7/9 Hinglish), and no note was added to the 2 queries that have none. In two of the misses the right note was the closest but just below the threshold, so nothing was added; for *"festival shopping"* (in both languages) the closest note was the Goa trip note, also below the threshold, so nothing was added either.

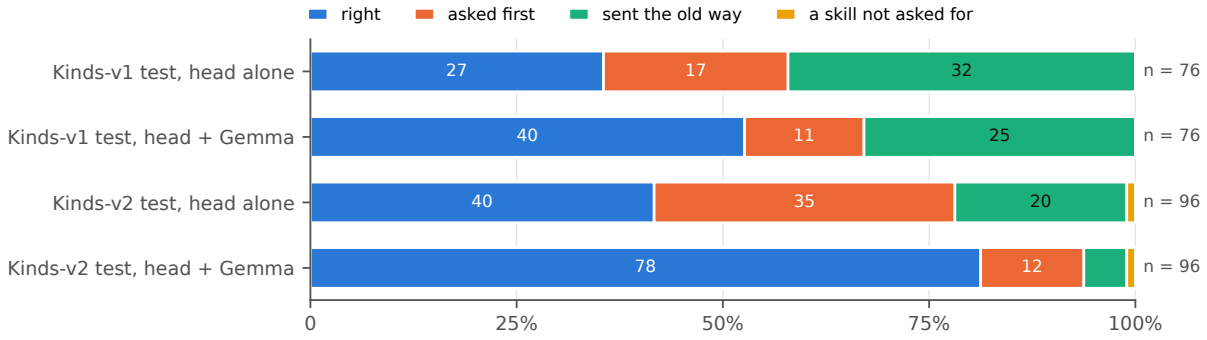


Figure 4: What happened to requests for a skill on the test splits, with the classifier alone and with the language model's second opinion.

6.3 Answering from the notes

On the test split, all 19/19 answerable questions were answered with the right facts, and all 6/6 unanswerable ones got the "not in your notes" reply; generating an answer took 4.9 s on average. On development data 4/6 unanswerable questions were declined: twice the model said in its own words that the notes do not give a cake's price, instead of the exact marker. The language of the answer is weaker than its content: in the test run (before a fix), 4/10 answers to Hinglish questions were Hinglish; with the fix, 9/12 on development data.

6.4 Code

Three models already on the laptop were compared on development data. After the checks were reviewed, Gemma-4-E2B and Qwen3-4B were both right 22/24 times, but Gemma's generation took 11.1 s per request against 29.0 s and needs no second model in memory, so Gemma does code too. On the test split, run once, Gemma was right 21/24 times: every function, fix and explanation, but only 3/6 scripts. The failed scripts kept reading input until the input ended (and crashed), stopped at the first empty line, or reversed the letters of a line instead of its words. Generation took 8.9 s per request on average. The sandbox found no error in any answer, so the retry loop never ran. Two weaknesses that the scores do not show: the three English requests to explain code were answered in Hinglish, and one explanation stated a wrong output and still passed, because explanations are judged by the facts they must contain.

Table 4: Code requests right, by type (6 per type), and generation seconds per request.

Model and run	function	script	fix	explain	all	gen. s / request
Gemma-4-E2B, first harness and checks (dev)	6/6	1/6	6/6	6/6	19/24	11.1
Qwen3-1.7B, first harness and checks (dev)	5/6	1/6	5/6	6/6	17/24	10.4
Qwen3-4B, first harness and checks (dev)	6/6	5/6	6/6	6/6	23/24	29.0
Gemma-4-E2B, final harness (dev)	6/6	5/6	6/6	6/6	23/24	9.1
Gemma-4-E2B, final harness (**test**, once)	6/6	3/6	6/6	6/6	21/24	8.9

6.5 Acceptance run

Before the freeze, every goal feature was used end to end on a scratch copy of the service with sample notes: the same requests a user would type, over the same socket the terminal client uses, with yes/no questions answered. 34/36 requests passed. The browser page was driven step by step in Chrome, headless and in a visible window, and passed 8/8 and 8/8 steps (load, a Try button, a task made and listed, the page's own yes/no question, a code box, the Copy button read back

from the clipboard, a streamed chat answer with its caveat). A reminder due at 17:31 popped up at 17:31:02. The automated tests passed: 448 Rust tests, 510 Python tests, 6 sandbox tests and 4 model-worker tests.

The two failed requests: *"how much is my rent?"* was refused, because both the classifier and Gemma took it for an app request (asked as a notes question, the same answer component gets it right); and *"which preferences have i set?"* found nothing, because the memory search matches the stored words.

Table 5: Seconds until the whole answer, from the acceptance run: one run on the laptop as found, not a benchmark.

Request	s	Request	s
add a task with a time	0.3	translate into Hindi	2.8
add a task, Hinglish	4.8	translate into English	3.2
keyword search	0.3	rewrite politely	3.6
search by meaning	0.3	email (Hinglish / English)	8.5 / 10.8
search by meaning, Hinglish	2.6	answer from notes	9.4
summary of a note	7.9	write a function	14.0
chat: a short poem	10.3	fix a function	4.3
chat: an explanation	23.2	a very long essay (cut off)	58.3

7 Known limitations

These 12 were written down at the freeze and not fixed, because the author froze the product:

1. Some notes questions are refused (*"how much is my rent?"*); others are answered (*"maa ka birthday kab hai?"*).
2. *"which preferences have i set?"* finds nothing, although a preference is stored.
3. *"write a program to solve X"* goes to chat: the whole program comes, but without the sandbox check.
4. Rewrite requests are often asked about first.
5. Scripts that read input to its end are Gemma's weak spot.
6. Jokes are answered by chat; *"what time is it in London"* goes to notes.
7. Markdown in chat answers (bold, bullets) is shown as typed.
8. Answers from notes to Hinglish questions are sometimes English.
9. Code may use Python's standard library only, and is checked by import or compile, never run with input.
10. Once, code did not show on the author's page; this could not be reproduced.
11. The page can be shared on the wifi over plain HTTP when the service is installed that way (off now).
12. There is no memory of the conversation: *"continue"* or a follow-up does not see the last answer.

In addition, every evaluation set is small and synthetic, about an imagined user's life; real use will find other mistakes. Energy use was not measured.

8 Installing and using it

Setup follows the repository's README: a Python environment, the models fetched and checked, the encoder converted, the heads trained (for v3 also the kind head, `python experiments/v3/kind_head.py --dataset kinds_v2`), and a release build with the `llama` feature. Then:

- `scripts/install_service.sh ~/notes` installs the always-on service for a notes folder; `scripts/install_reminders.sh` adds the morning reminder.
- `kestrel open` opens the page; `kestrel chat` starts a conversation in a terminal; `kestrel say "mere tasks btao"` sends one request; `kestrel stop` frees all of its memory.

The installed v3 service runs from its own checkout (a git worktree) at the tested build, commit `86cb3a7`, so the research can switch branches in the main checkout without touching it.

9 What comes next

- **Learn from real mistakes.** The next round starts from the author's own misrouted requests: new kinds and code sets built from them, reviewed, with a fresh test split.
- **A bigger machine (v4).** Larger models need more memory and speed than this laptop has; on it, Qwen3-4B already writes only 8.2 tokens per second.
- **The research first.** From the week of 1 November 2026 the pre-registered research (personalization and quantization experiments, final statistics) continues; heavy runs of the two never overlap.

A Decision record

Decision	What was decided
ADR-036	v3's goal on this laptop, fully offline; v4 is a bigger machine.
ADR-037	A kind decision before the research cascade; the first kinds data set.
ADR-038	Chat answers timeless explanations with a visible caveat; more families before review.
ADR-039	Notes found by meaning after the keyword search, in the app only.
ADR-040	Hinglish queries go through English; only the closest note is added.
ADR-041	Notes Q&A: a *notes* kind; answers only from the notes, grounded, with "not in your notes".
ADR-042	After the notes review: freeze the notes sets, test once on Kinds-v1, more notes families.
ADR-043	v3 becomes the always-on service, run from its own worktree.
ADR-044	Qwen3-4B for code (superseded by ADR-046); the code loop retries only on the sandbox's own errors; one review for the code and Kinds-v2 sets.
ADR-045	The author waives that review; the AI assistant reviews with an independent agent.
ADR-046	Gemma writes and fixes code too (one model in the service).
ADR-047	The page may be shared on the wifi (off by default).
ADR-048	v3 frozen after the acceptance run (tag `v3-frozen`).

B Where the numbers come from

Every measured value, share and table entry in this report comes from the committed logs through the script `experiments/analysis/paper_assets_v3.py`; its tests fail if a document asks for a value the script does not generate. Configuration constants (sandbox limits, retries, the number of notes given) and a few small counts in the prose (such as the misrouted requests named in Section 6) are typed and were checked by hand. The logs are under `experiments/logs/v3/`: `kinds/`, `notes/`, `code/`, `writer/`, `mmap/`, `worker/` and `acceptance/`; the laptop's description is

in `experiments/PREREG.md` and its model speeds in `experiments/logs/E0/`. `paper/v3/number_check.md` records an independent check of every number.